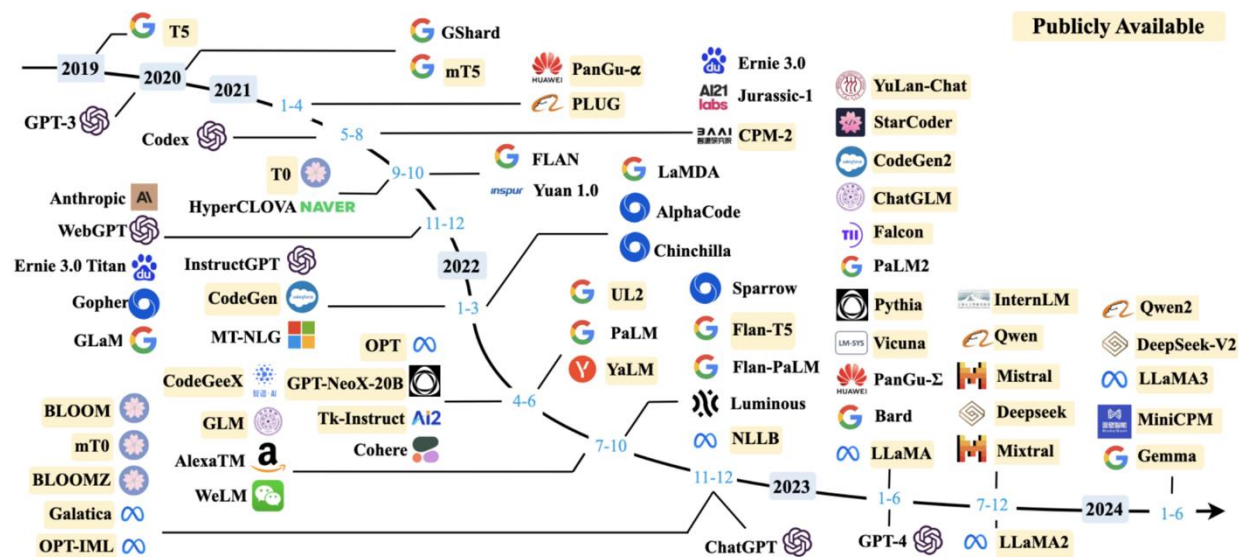


Large Language Model Pretrain

Mengyue Yang

Recap: The Vanilla Transformer

- In Lecture 1, we learned the **Transformer** architecture (Vaswani et al., 2017).
- Key components: Self-Attention, Feed-Forward Networks (FFN), Positional Encodings.
- *Question:* How does a mathematically empty structure become a knowledgeable AI?
- *Answer:* Through massive **Pretraining**.



What is Pretrained Foundation Models

Definition (Min et al., 2023)

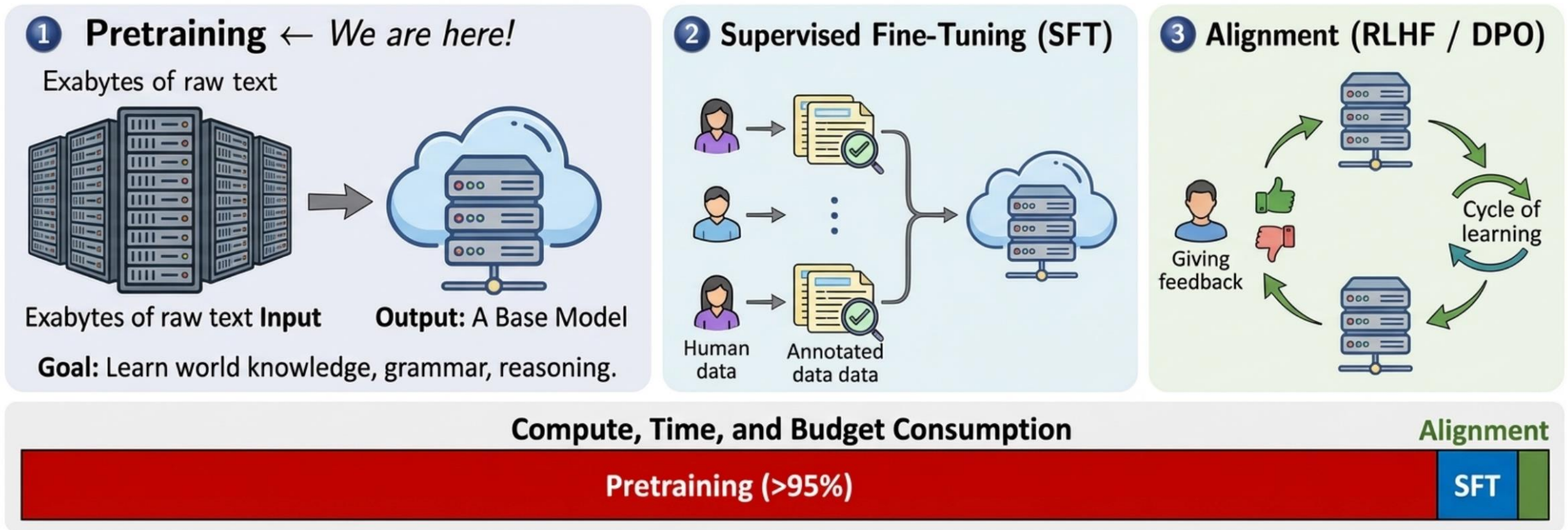
Pretrained Foundation Models (PFMs) are powerful general models that are effective in various fields or across fields, demonstrating great potential in learning feature representations in various tasks.

They cover three major AI fields:

- 1 Natural Language Processing (NLP) - e.g., GPT, Llama
- 2 Computer Vision (CV) - e.g., ViT
- 3 Graph Learning (GL) - e.g., Graphormer

The LLM Lifecycle

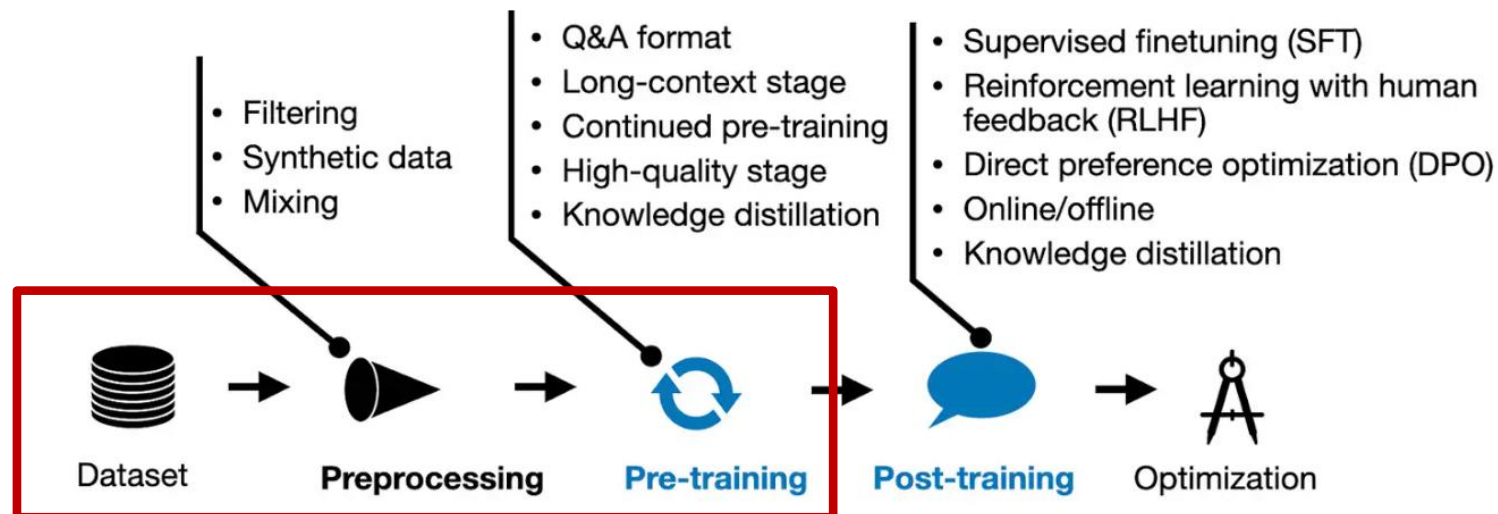
Building a modern chatbot (like ChatGPT) involves several stages:



Anatomy of an LLM Pretraining Run

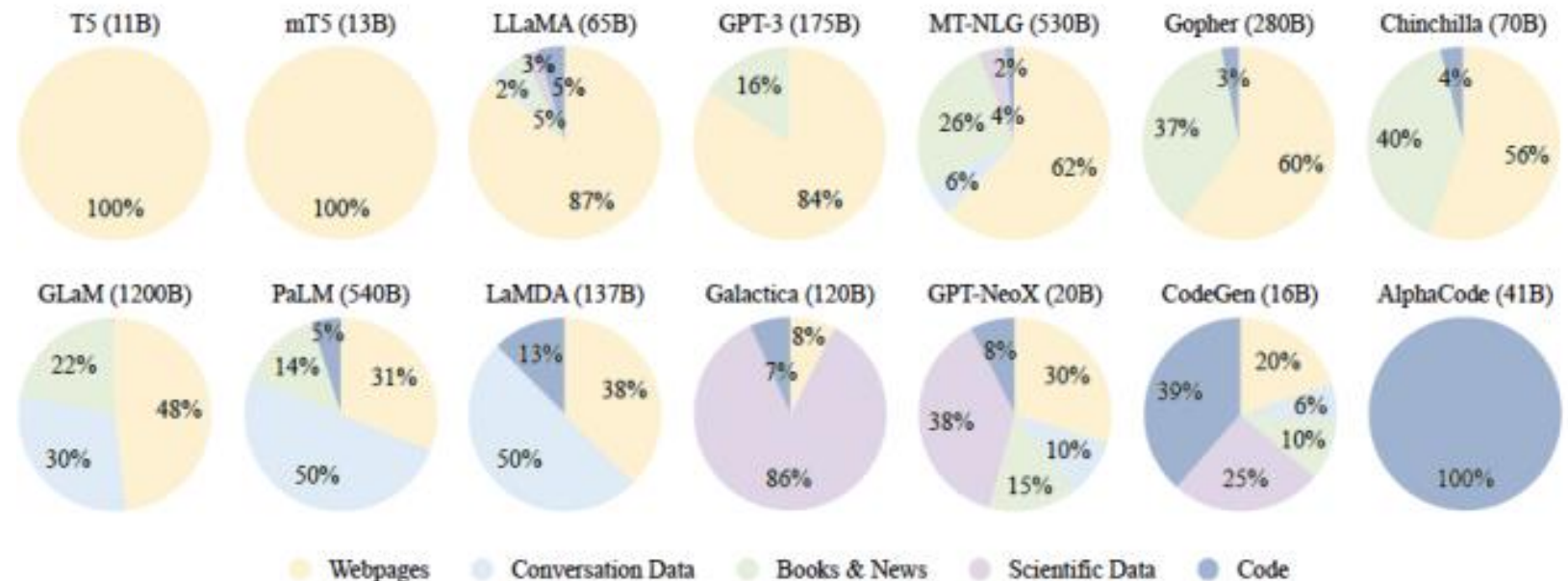
To pretrain a modern LLM, you need 4 core pillars:

- 1 **Data Pipeline:** The fuel (Trillions of tokens).
- 2 **Objectives & Architecture:** The engine (CLM, RoPE, SwiGLU).
- 3 **Scaling Laws:** The physics (Compute-optimal rules).
- 4 **Training Dynamics:** The infrastructure (BF16, AdamW, Parallelism).



Module 1: The Data Pipeline

- The secret sauce of modern LLMs (Llama 3, Qwen 2) is not a radically new architecture, but **data quality and scale**.
- Pretraining data volume has grown from 3.3 Billion tokens (BERT) to over 15 Trillion tokens (Llama 3).
- *Garbage In, Garbage Out*: A model trained on noisy data will hallucinate and generate toxic content.



The Landscape of Data Sources

Modern LMs are trained on a mixture of different text domains:

Source	Characteristics	Example
Web Crawl	Massive volume, high noise, diverse	Common Crawl
Books	Long-context, narrative, high-quality	BookCorpus
Wikipedia	High knowledge density, factual	English Wiki
Code	Logical, structured, high reasoning	GitHub
Academic	Scientific reasoning, math	ArXiv, PubMed

Data from Webpages

- **Massive Data Source:** Webpages constitute the largest share of training data for LLMs, with platforms like Common Crawl providing petabytes of raw internet content collected over years.
- **The Quality Challenge:** While these datasets contain valuable information (e.g., Wikipedia), they are heavily contaminated with HTML code, boilerplate, SEO spam, and toxic content.
- **Filtering & Curation:** Rigorous data cleaning is essential to improve quality, leading to the development of curated, open-source datasets like C4, The Pile, and RefinedWeb for model training.



Why is Code Data Crucial?

- Even if you don't want your LLM to write Python, you **must** train it on code.
- **Why?**
 - Code is strictly logical and structured.
 - Training on code drastically improves the **general reasoning and logic capabilities** of the model on natural language tasks.
- Usually comprises 5% to 15% of the total pretraining mixture.

The "Dirty Work": Data Processing Pipeline

Transforming Common Crawl into training data drops up to **80%-90%** of the raw bytes.

- ➊ **Parsing & Extraction:** HTML to raw text.
- ➋ **Quality Filtering:** Heuristics and Classifiers.
- ➌ **Deduplication:** Exact and Fuzzy.
- ➍ **Decontamination:** Benchmark removal.
- ➎ **PII Removal:** Privacy protection.

Step 1 & 2: Extraction and Quality Filtering

Heuristic Filtering:

- Remove documents with too few words or unnatural word lengths (e.g., "aaaaahhhhh").
- N-gram repetition filters to remove spam.

Classifier Filtering (Machine Learning):

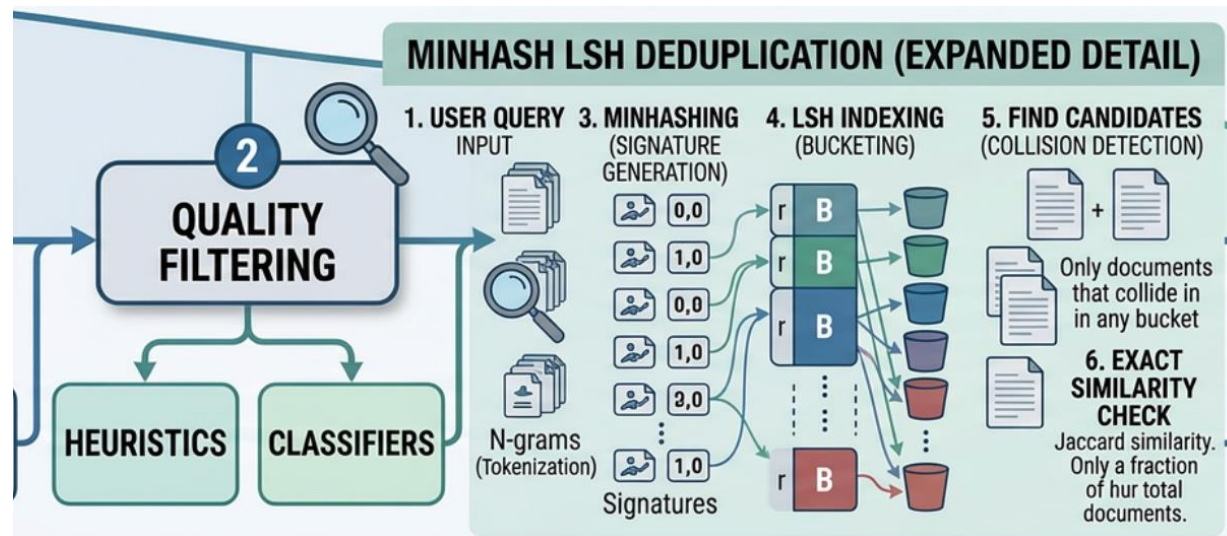
- Train a lightweight classifier (e.g., FastText) to distinguish between "Wikipedia-like" high-quality text and "Spam" text.
- Discard documents with low quality scores.

Step 3: Deduplication (Why is it so important?)

- The internet is full of duplicate content (copies of Wikipedia, repeated news).
- **The Danger of Duplicates:**
 - If an LLM sees a text 1,000 times, it will **memorize** it rather than learn generalized patterns.
 - Memorization wastes precious GPU compute.
 - Leads to privacy leaks (the model might spew exact training data verbatim).

Exact vs Fuzzy Deduplication

- **Exact Match:** Simple cryptographic hash (SHA-256) of the document. Removes identical files.
- **Fuzzy Match:** What if a news article has a different date or one extra paragraph? (Near-duplicates).
- **Solution: MinHash LSH (Locality-Sensitive Hashing).**
 - Efficiently finds documents that share high n-gram overlap in a dataset of billions of documents without $O(N^2)$ comparisons.



Step 4 & 5: Decontamination and PII

Decontamination:

- We evaluate LLMs on benchmarks (e.g., MMLU, GSM8K).
- If the test questions are in the pretraining data, the model is "cheating".
- We must scan and remove benchmark datasets from the training corpus.

PII Removal (Personally Identifiable Information):

- Use Regex and NER (Named Entity Recognition) to detect and mask phone numbers, emails, and SSNs.

Tokenization: From Text to Numbers

- Neural networks cannot process characters directly; they process vectors of numbers.
- **Tokenization** bridges this gap.
- *Example:* "Pretraining is fun!" → [144, 55, 302, 19, 9]
- **Character-level:** Vocabulary is small (~ 100), but sequences become extremely long. Slows down Attention ($O(N^2)$).
- **Word-level:** Vocabulary is infinite. Out-Of-Vocabulary (OOV) problem for new words.
- **Subword-level:** The golden middle.
 - E.g., "Unbelievable" → ["Un", "believ", "able"].
 - Standard for all modern LLMs.

Algorithms: BPE and SentencePiece

Byte-Pair Encoding (BPE):

- Starts with individual characters/bytes.
- Iteratively merges the most frequent adjacent pairs.
- E.g., 't' + 'h' → 'th'. 'th' + 'e' → 'the'.
- Used by GPT-4, Llama, RoBERTa.

SentencePiece:

- Treats spaces as a standard character (e.g., ' ').
- Great for languages without spaces (Chinese, Japanese).

Vocabulary Size Trade-offs

- **Small Vocab (e.g., 32K in Llama 2):**
 - Pros: Smaller embedding matrix, saves memory.
 - Cons: Longer token sequences for the same text (slower inference).
- **Large Vocab (e.g., 128K in Llama 3, 100K in GPT-4):**
 - Pros: Better compression, highly efficient for Multilingual text.
 - Cons: Massive embedding matrix (requires more parameters).

Modern Considerations: Multilingual Numbers

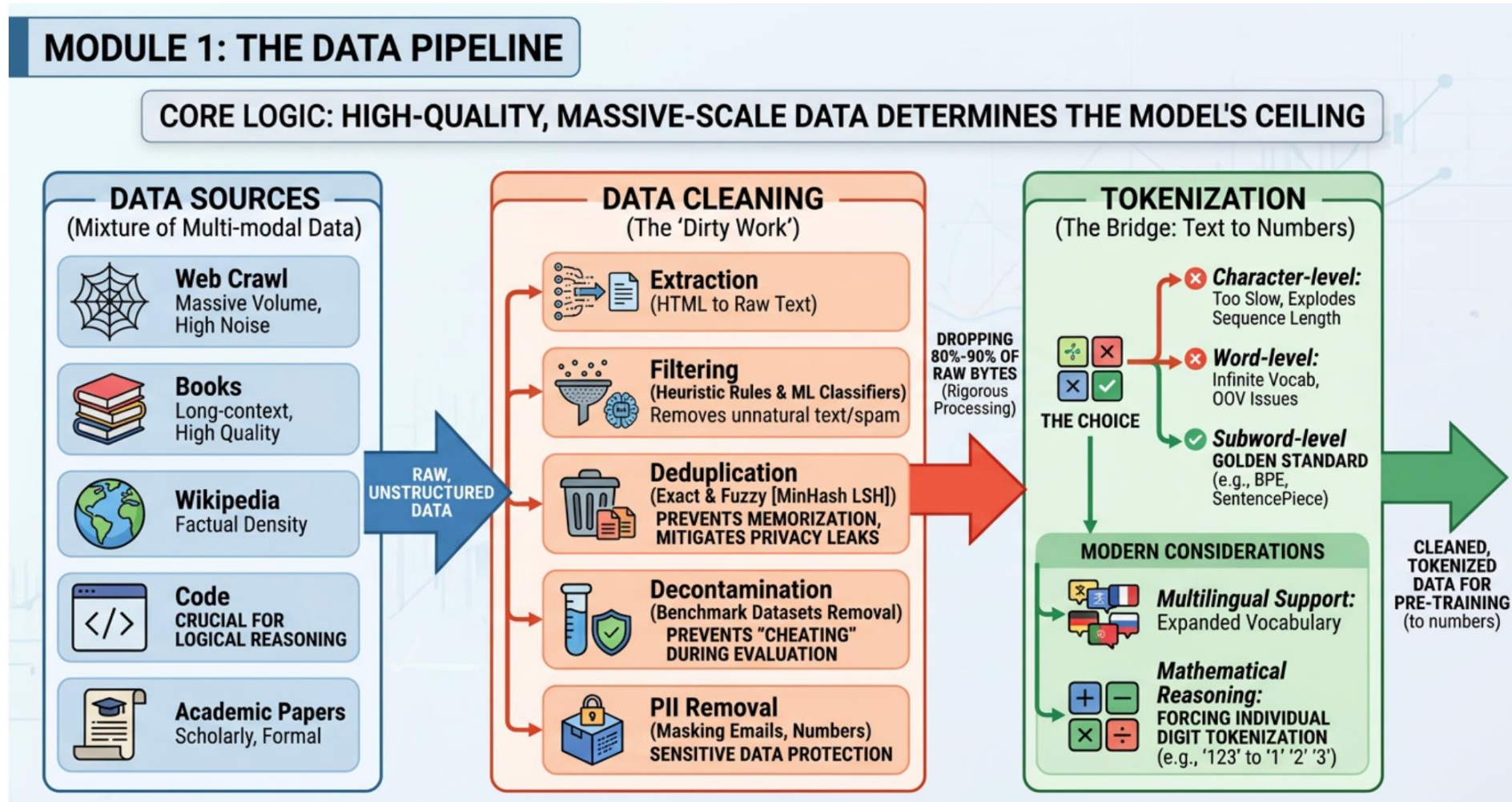
Multilingual Support:

- Older models (like GPT-2) heavily fragmented non-English text (e.g., 1 Chinese character = 3 tokens).
- Modern tokenizers allocate more vocabulary to diverse languages to ensure fairness and efficiency.

Handling Numbers:

- Historically, "12345" might be tokenized as ["123", "45"].
- This destroys arithmetic logic.
- Modern models force **individual digit tokenization** (["1", "2", "3", "4", "5"]) to improve math reasoning.

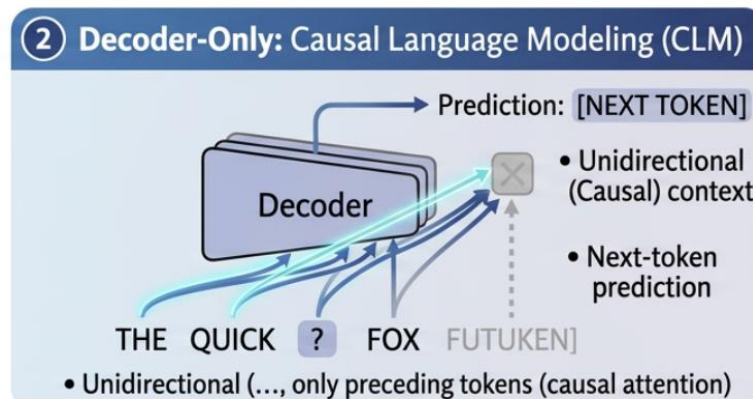
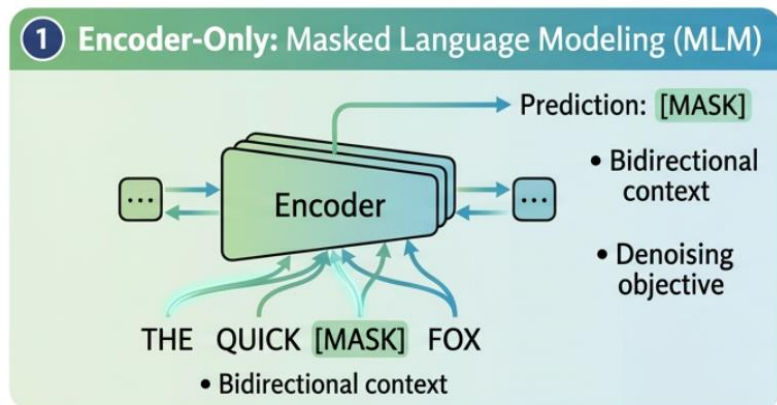
Module 1: Overall Data Pipeline



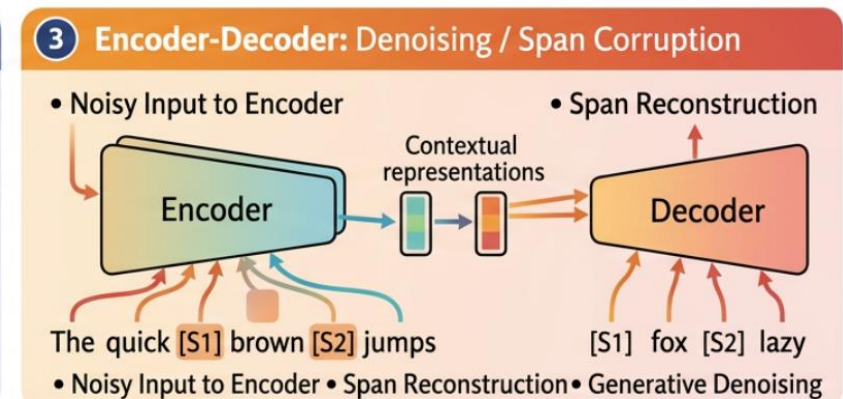
Module 2: Objectives & Architecture

We have tokens. How does the model learn from them without human labels? (Self-supervised learning). Three historical paradigms:

- 1 **Encoder-Only:** Masked Language Modeling (MLM).
- 2 **Decoder-Only:** Causal Language Modeling (CLM).
- 3 **Encoder-Decoder:** Denoising / Span Corruption.

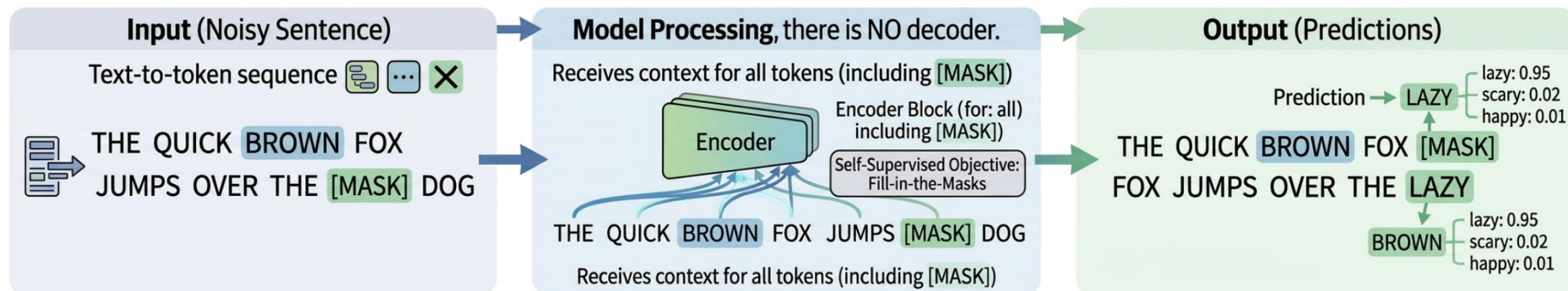


KEY = Self-supervised training to compensate denoising of processor full plant.



Encoder-Only: Masked Language Modeling (MLM)

- **Representative Model:** BERT, RoBERTa.
- **Task:** "Fill in the blank" (Cloze task).
- **Mechanism:** 15% of tokens are masked. Model uses bidirectional context (left and right) to predict them.
- **Input:** "The quick brown [MASK] jumps over the [MASK] dog."
- **Pros:** Deep bidirectional understanding. Excellent for text classification and Embeddings.
- **Cons:** Terrible at generating text (not auto-regressive).



Encoder-Only: Masked Language Modeling (MLM)

Specific Example:

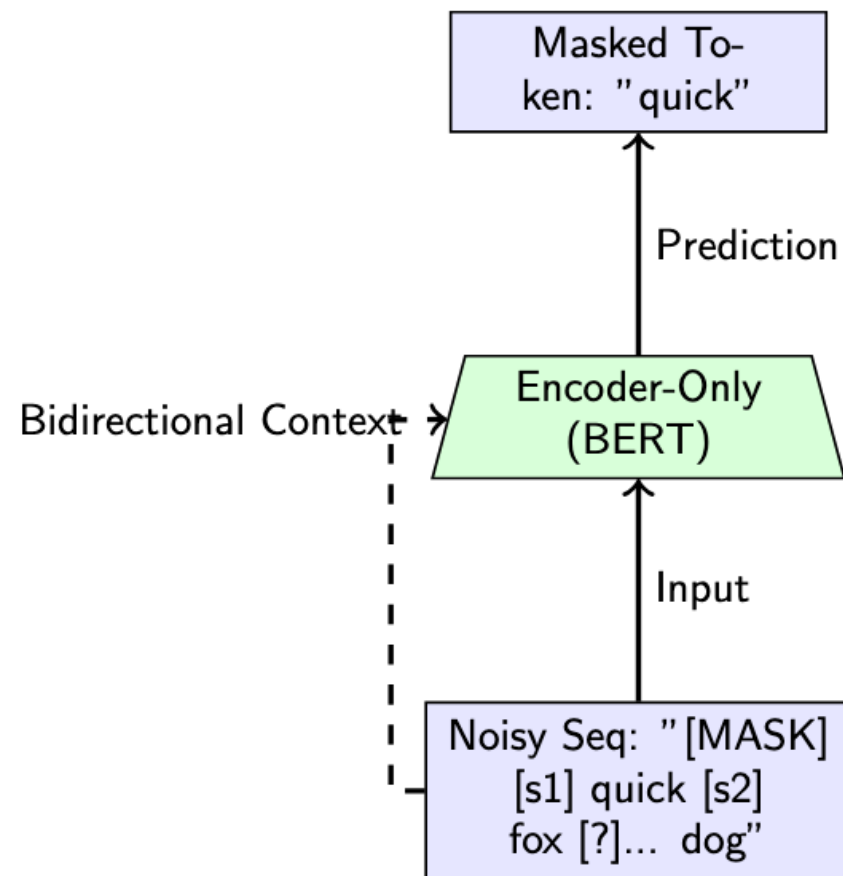
- **Input:** "The [s1] brown [s2] jumps [?] dog"
- **Masked Spans:**
 - 1 '[s1]': 'quick'
 - 2 '[s2]': 'fox'
 - 3 '[?]'': 'lazy'
- **Prediction:** 'P("fox" — Noisy Context)'

Optimization Objective:

$$L_{MLM} = - \sum_{[MASK] \in S_{mask}} \log P(x_i | \text{Full Seq } S_{noisy})$$

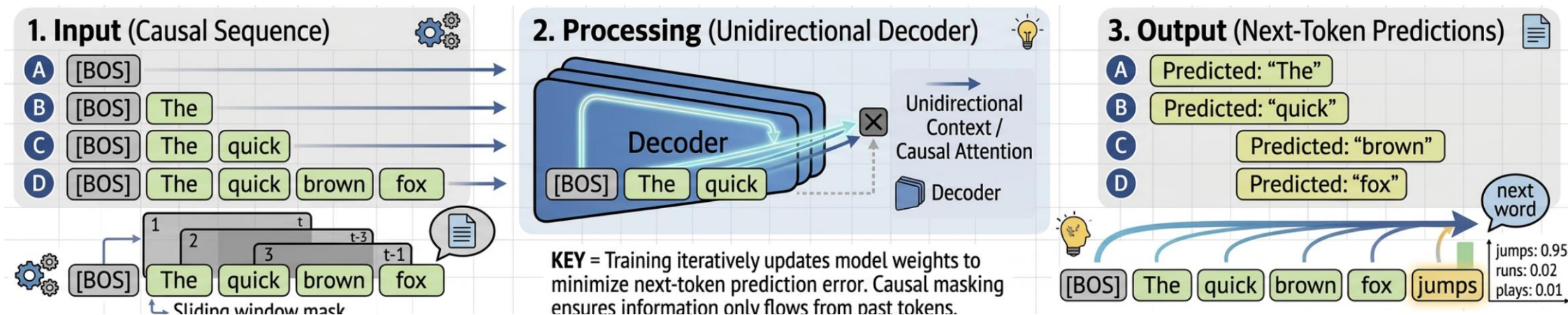
where S_{noisy} includes *all* surrounding tokens.

Significance: Learns deep **Bidirectional Context**.
Excel at *understanding* tasks.



Decoder-Only: Causal Language Modeling (CLM)

- **Representative Model:** GPT, Llama, Qwen, Mistral.
- **Task:** Next-token prediction.
- **Mechanism:** Predict x_t given $x_1, x_2, \dots, x_{t-1}$.
- **Input:** "The quick brown fox jumps over the" → *Target:* "lazy"
- **Pros:** Naturally aligned with text generation tasks. Scales incredibly well.



Decoder-Only: Causal Language Modeling (CLM)

Specific Example:

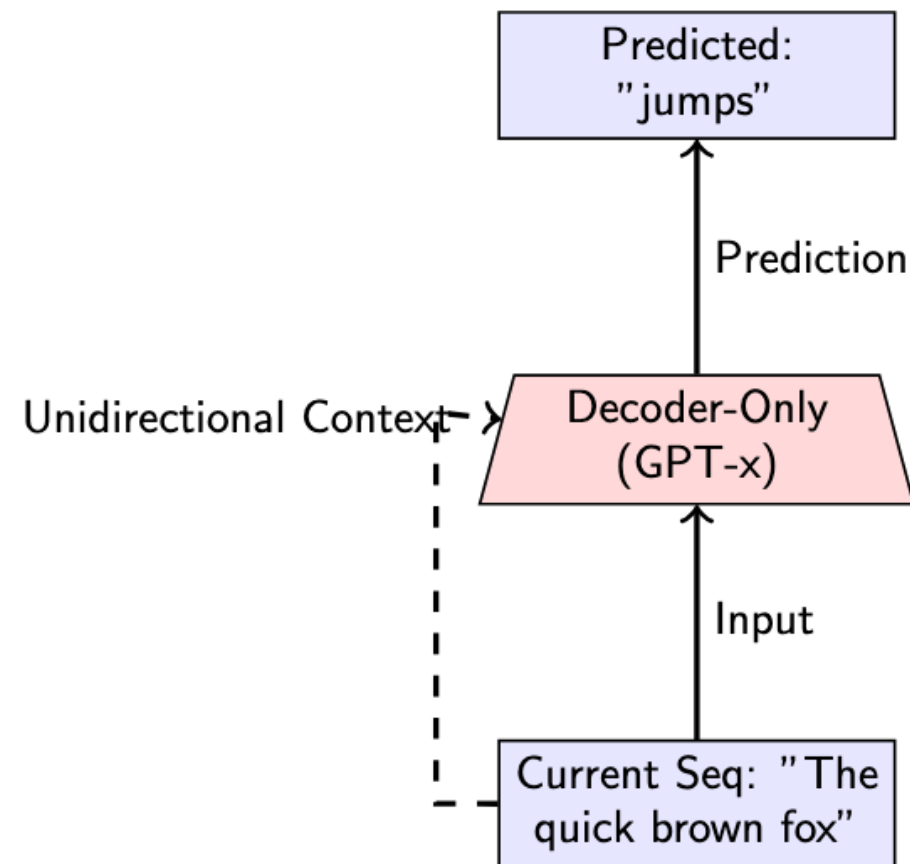
- **Current State:** '[BOS]', 'The'
- **Target:** 'P("quick" — [BOS], "The")'
- **Next-Token Prediction:**
 - '[BOS]' → predicts 'The'
 - '[BOS]', 'The' → predicts 'quick'
 - '[BOS]', 'The', 'quick' → predicts 'brown'

Optimization Objective:

$$L_{CLM} = - \sum_{i=1}^T \log P(x_i | x_{<i})$$

where $x_{<i}$ are *only* preceding tokens.

Significance: Learns strong **Generative Continuity**.
Unparalleled at *creating* coherent text.



Recap: Causal Masking

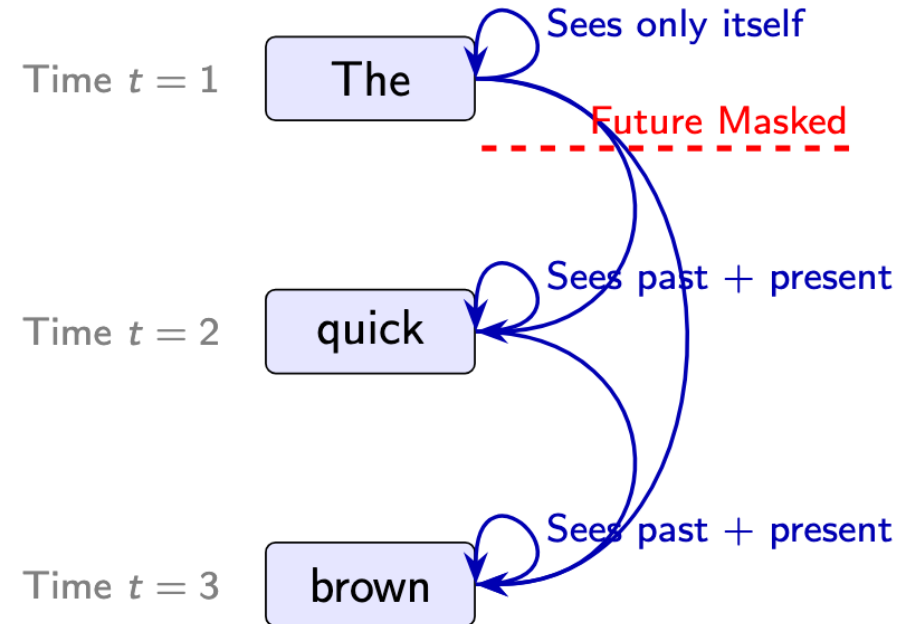
The Problem: Parallel Training vs. Sequential Generation

- **Inference (Generation):** The model generates text token-by-token. It only knows the *past*.
- **Training:** For efficiency, we feed the entire sequence into the model at once (in parallel).
- **The Risk ("Cheating"):** Without masking, the Self-Attention mechanism would allow token i to "look ahead" at token $i + 1$ to predict token $i + 1$, completely ruining the learning process.

The Solution: Causal Masking

Forces the attention mechanism to be strictly **Unidirectional**. Information can only flow forward in time.

Information Flow in Self-Attention



Recap: Causal Masking

The Math Behind the Mask

Standard Self-Attention adds a Mask Matrix M before applying the Softmax function:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + \mathbf{M} \right) V$$

How it works:

- **Lower Triangle (0):** Past/Current tokens. Adding 0 doesn't change the QK^T score.
- **Upper Triangle ($-\infty$):** Future tokens. Adding $-\infty$ makes the exponential in the Softmax drop to exactly 0.
- $e^{-\infty} = 0 \rightarrow$ Future token weights become zero!

Causal Mask Matrix (M)

Keys (K) / Context to Attend To $\rightarrow$

	0	$-\infty$	$-\infty$	$-\infty$
$\uparrow$ Queries (Q)	0	0	$-\infty$	$-\infty$
	0	0	0	$-\infty$
	0	0	0	0

Encoder-Decoder: Denoising (Prefix LM)

- **Representative Model:** T5, BART.
- **Task:** Corrupt a sequence, ask the decoder to reconstruct it.
- **Input to Encoder:** "The quick brown [X] jumps over the [Y] dog."
- **Target for Decoder:** "[X] fox [Y] lazy"
- **Status Today:** Less popular for extreme scaling ($> 100B$ params) compared to pure Decoder-only models, but still highly effective for translation and summarization.

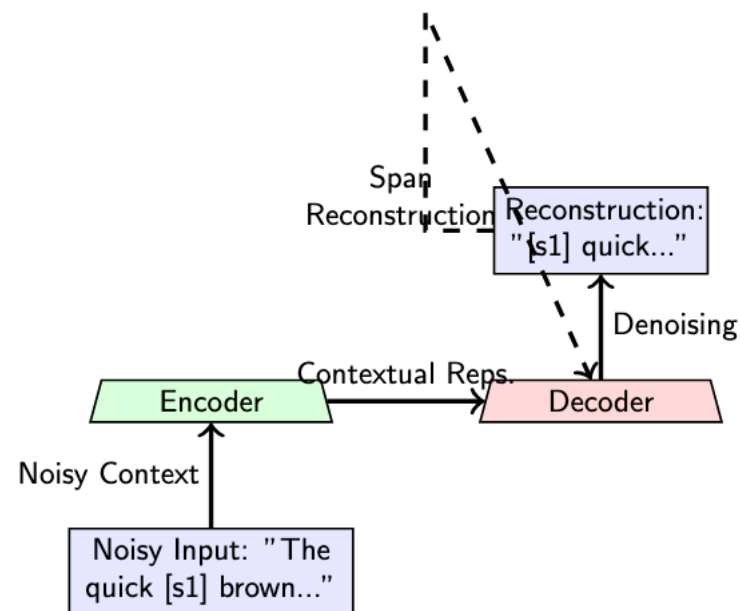
Encoder-Decoder: Denoising (Prefix LM)

Specific Example:

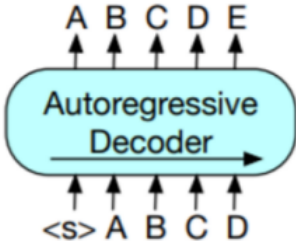
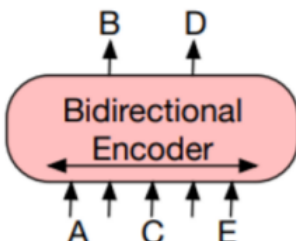
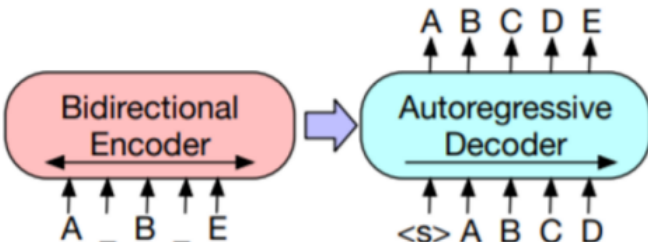
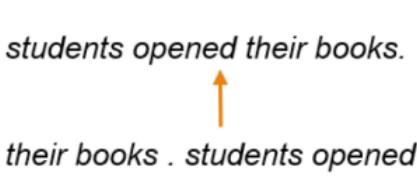
- **Noisy Input:**
"The [s1] brown [s2] jumps [?] dog"
- **Target Output:**
"[s1] quick [s2] fox [?] lazy"
- **Decoder Generation:**
 1. Predicts '[s1]', 'quick'.
 2. Then '[s2]', 'fox'.

Optimization Objective:

$$L_{S\&C} = - \sum_{j=1}^M \sum_{i=1}^{T_j} \log P(x_{j,i} | X_{\text{noisy}}, x_{j,<i})$$



Architecture

	Autoregressive language model (e.g., GPT, GPT-2/3)	Masked language model (e.g., BERT, RoBERTa, XLM-R)	Encoder-Decoder (e.g., BART)
Architecture illustration			
Training objective	Predicting what word comes next given previous words	Predicting masked words given other words in the sequence	Corrupting a sequence and then predicting the original sequence
Example			

Summary of Historical LMs (from Min et al., 2023)

Model	Architecture / Objective	Corpus Size	Parameters
BERT (2018)	Encoder / MLM	3.3B tokens	340M
T5 (2019)	Enc-Dec / Denoising	750 GB	11B
GPT-3 (2020)	Decoder / CLM	300B tokens	175B
Gopher (2021)	Decoder / CLM	300B tokens	280B

Modern Architecture: The "What Else"

- The original 2017 Transformer ("Attention is All You Need") is **obsolete** for modern LLM training.
- Scaling to billions of parameters caused extreme instability and memory bottlenecks.
- We must understand the 3 modern upgrades (used in Llama 3, Qwen 2, etc.):
 - ① **RoPE** (Rotary Positional Embedding)
 - ② **SwiGLU** (Activation Function)
 - ③ **RMSNorm & Pre-Norm** (Normalization)

Rotary Positional Embedding

- It balances absolute position (via rotation angle) and relative position (via dot product).
- **Extrapolation:** RoPE allows models to gracefully handle context lengths longer than they were trained on (e.g., via NTK-aware scaling or YaRN).
- It is the default for Llama, Qwen, Mistral, and PaLM.

Activation Functions: From ReLU to SwiGLU

- It balances absolute position (via rotation angle) and relative position (via dot product).
- **Extrapolation:** RoPE allows models to gracefully handle context lengths longer than they were trained on (e.g., via NTK-aware scaling or YaRN).
- It is the default for Llama, Qwen, Mistral, and PaLM.
- Unlike standard activations applied to a single linear layer, SwiGLU uses a **gating mechanism**.
- $\text{SwiGLU}(x) = (\text{Swish}(xW_1)) \otimes (xW_2)$
- It requires 3 weight matrices instead of 2.
- **Why use it?** Consistently yields better performance (lower perplexity) across all model scales, justifying the slight increase in compute.

Normalization: Post-Norm vs Pre-Norm

- **Vanilla Transformer (Post-Norm):** Normalization happens *after* the residual addition.
- **Problem:** In very deep networks (100+ layers), this causes severe gradient vanishing/exploding.
- **Modern LLMs (Pre-Norm):** Normalization happens *before* the Attention/FFN layer, inside the residual branch.
- **Result:** The main residual stream acts as an unobstructed highway. Drastically improves training stability.

RMSNorm and LayerNorm

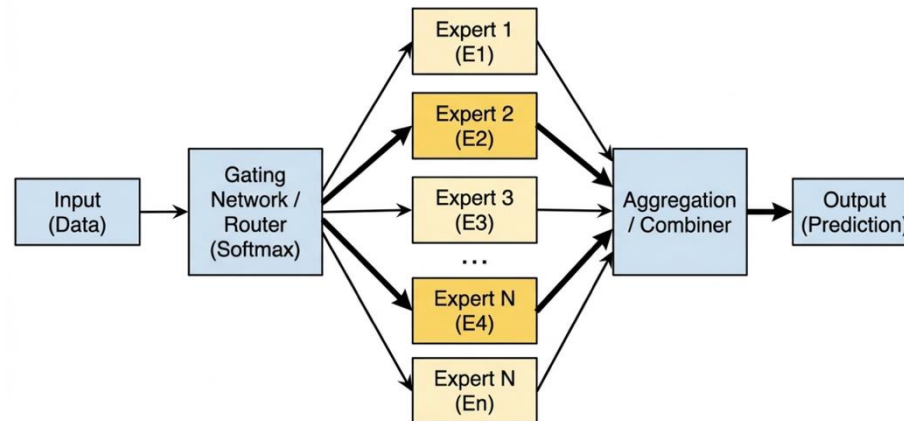
- **LayerNorm:** Centers the mean to 0 and scales the variance to 1.
 - Mean centering requires a global sum across features, which is slow on GPUs.
- **RMSNorm (Root Mean Square Normalization):**
 - Drops the mean centering entirely.
 - Scales only by the Root Mean Square.
- **Why RMSNorm?** Studies show mean centering offers no performance gain in Attention networks. RMSNorm is 10%-50% faster.

Grouped-Query Attention (GQA)

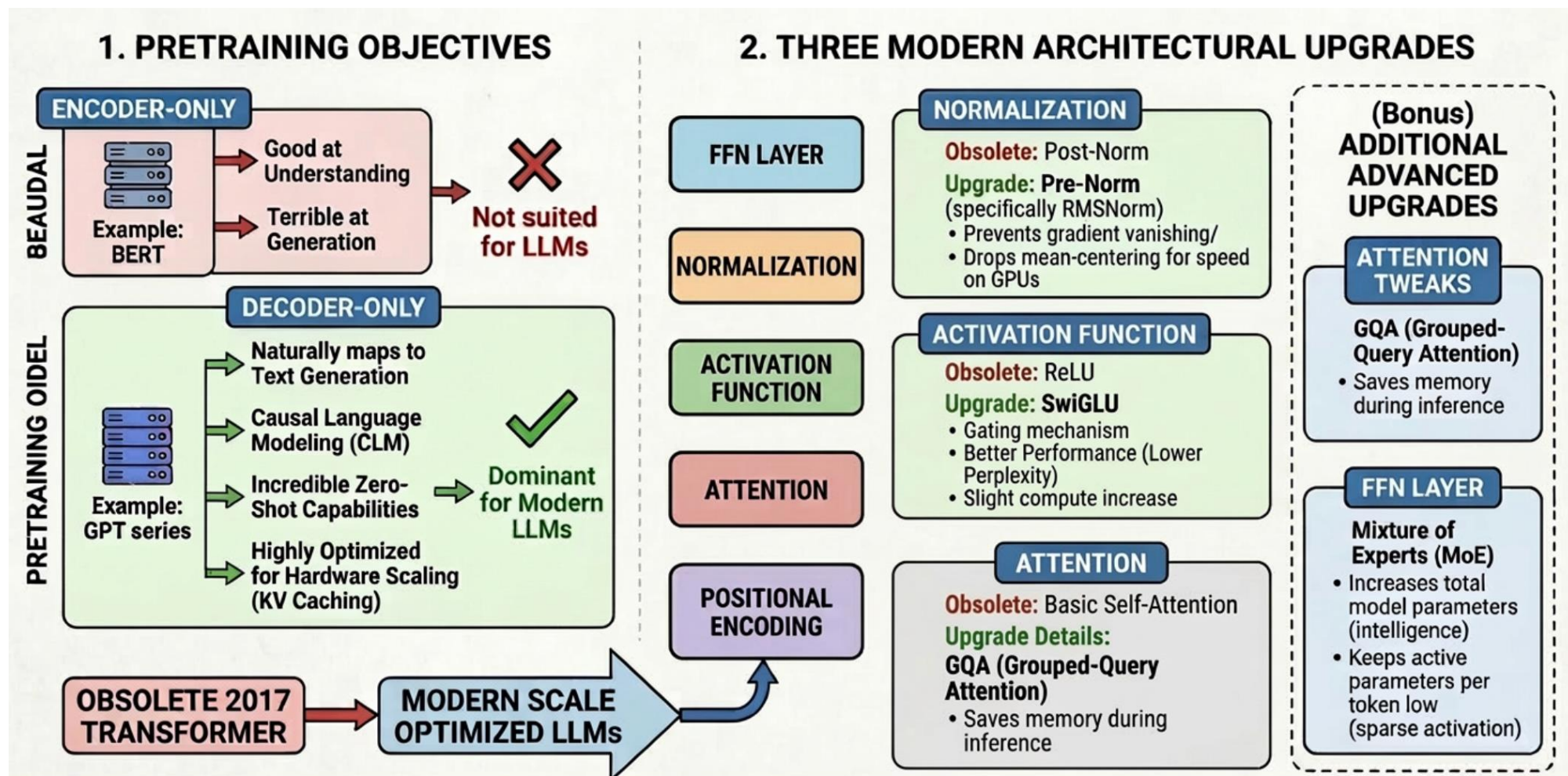
- **Problem with Multi-Head Attention (MHA):** During generation, storing the Key and Value matrices (KV Cache) consumes massive GPU memory.
- **Multi-Query Attention (MQA):** All Query heads share exactly 1 KV head. (Saves memory, loses quality).
- **GQA (The Sweet Spot):** Divides Query heads into groups (e.g., 8 Q heads share 1 KV head).
- Standard in Llama 2/3 and Mistral for fast, memory-efficient inference.

Advanced Concept: Mixture of Experts (MoE)

- How to increase model intelligence without making it too slow to run?
- **MoE:** Replace the dense FFN layer with multiple smaller FFNs ("Experts").
- **The Router:** A small neural network dynamically routes each token to the top- K (usually top-2) most relevant experts.
- **Sparse Activation:** A 47B parameter model (like Mixtral 8x7B) might only use 13B parameters during inference for any single token.



Module 2: Overall Objective and Architecture



Module 3: Scaling Laws

-How much data do we need to train LLM

- Training a 70B model costs millions of dollars. You cannot afford to guess hyperparameters.
- **Scaling Laws** provide mathematical formulas predicting model performance (Loss) based on three variables:
 - 1 Compute (C) - FLOPs
 - 2 Dataset Size (D) - Tokens
 - 3 Model Parameters (N)
- Loss follows a predictable power-law curve.

Kaplan's Scaling Laws (OpenAI, 2020)

- Found that model performance scales predictably.
- **Key conclusion:** "Model size (N) matters most. Bigger models are always better, even if trained on relatively less data."
- **Result:** This led to the creation of massive models like GPT-3 (175B parameters), which was trained on only 300B tokens.

The Chinchilla Paradigm (DeepMind, 2022)

- Hoffmann et al. found a flaw in Kaplan's setup.
- They discovered that GPT-3 and similar models were **severely undertrained**.
- They proposed the **Compute-Optimal** scaling rule.
- Chinchilla Rule: If you increase your Compute budget by 10x, you should scale Model Parameters (N) and Data (D) equally ($\approx 3.16x$ each).
- $N \propto \sqrt{C}$
- $D \propto \sqrt{C}$
- **Golden Ratio:** You should have roughly **20 tokens of data for every 1 parameter**.

The Chinchilla Paradigm (DeepMind, 2022)

How much data do you need for compute-optimal training?

Model Size (N)	Optimal Tokens ($D \approx 20N$)
1 Billion	20 Billion tokens
7 Billion	140 Billion tokens
70 Billion	1.4 Trillion tokens
175 Billion (GPT-3 size)	3.5 Trillion tokens (GPT-3 only had 0.3T!)

The Llama Era: Overtraining (Beyond Compute-Optimal)

- Chinchilla optimizes for the *training* budget.
- But what if we care about the **inference** budget? (We want small, fast models that are very smart).
- **Overtraining:** Meta trained Llama-3-8B on **15 Trillion tokens!**
- That is ≈ 1800 tokens per parameter (90x the Chinchilla limit).
- **Result:** Small models punching way above their weight class.

Module 4: Training Dynamics & Infra

- Training an LLM is not like training ResNet on a single GPU.
- It involves orchestrating thousands of H100 GPUs continuously for 3 to 6 months.
- Hardware fails. Network switches drop packets. Gradients explode.
- **Training Stability** is the most critical engineering task.

Optimization: AdamW

- Standard Stochastic Gradient Descent (SGD) is far too slow and unstable for LLMs.
- **AdamW (Adam with Weight Decay)** is the undisputed standard.
- It maintains moving averages of the gradients (momentum) and the squared gradients (variance) to adapt the learning rate per parameter.
- Weight decay prevents parameters from growing too large, acting as regularization.

Learning Rate (LR) Scheduling

- You cannot use a fixed learning rate.
- If the LR is too high initially $\rightarrow$ gradients explode, model diverges (Loss = NaN).
- If the LR is too low later $\rightarrow$ model stops learning and wastes compute.
- ① **Linear Warm-up:** Start LR at 0 and slowly increase it to LR_{max} over the first few thousand steps. (Allows the randomly initialized weights to settle).
- ② **Cosine Decay:** Slowly decrease the LR following a cosine curve down to $\sim 10\%$ of LR_{max} by the end of training. (Allows fine-grained settling into the local minima).

The Standard: Warm-up and Cosine Decay

- 1 **Linear Warm-up:** Start LR at 0 and slowly increase it to LR_{max} over the first few thousand steps. (Allows the randomly initialized weights to settle).
- 2 **Cosine Decay:** Slowly decrease the LR following a cosine curve down to $\sim 10\%$ of LR_{max} by the end of training. (Allows fine-grained settling into the local minima).

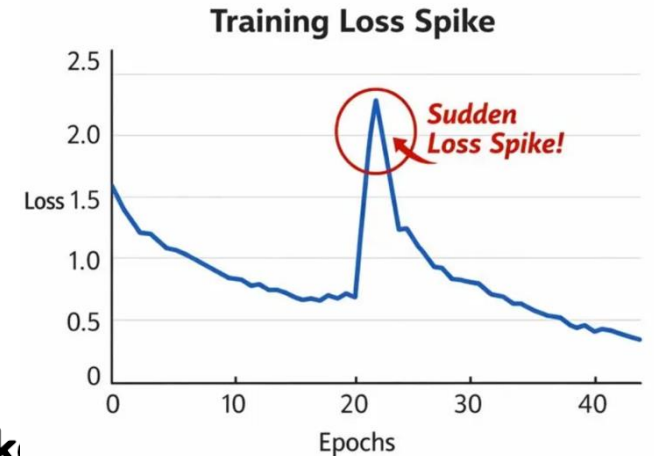
Batch Size Strategy

- Modern LLMs use enormous batch sizes (e.g., 4 million tokens per batch).
- **Dynamic Batch Sizing:** Start with a smaller batch size at the beginning of training (when gradients are noisy but highly informative).
- Gradually increase the batch size as training progresses to maximize GPU throughput and stabilize convergence.

Numerical Precision: The Memory Bottleneck

- Standard Deep Learning used **FP32** (32-bit floating point).
- Moving to 16-bit halves the memory requirement and doubles the math speed (using Tensor Cores).
- **Problem with FP16:** It has a very small dynamic range (max value ≈ 65504).
- LLM gradients often exceed this, causing **Overflow (NaN)**.
- Invented by Google (Brain Float 16).
- **FP16:** 5 bits for exponent, 10 bits for fraction.
- **BF16:** 8 bits for exponent, 7 bits for fraction.
- **Why it works:** BF16 has the *exact same dynamic range as FP32* (max value $\sim 10^{38}$).
- You lose a tiny bit of decimal precision, but you almost entirely eliminate gradient overflow. **BF16 is the standard today.**

The Nightmare: Loss Spikes



- You are on day 45 of training. The Loss curve is smoothly dropping.
- Suddenly, the Loss jumps from 2.0 to 9.5 in 10 steps. This is a **Loss Spike**
- **Causes:** A highly abnormal batch of data, subtle numerical instability, or a failing GPU memory module.
- **Solution Protocol:**
 - ① **Frequent Checkpoints:** Save the model weights every few hours.
 - ② **Rollback:** Stop training, reload the checkpoint from before the spike.
 - ③ **Skip Data:** Skip the data batches that caused the spike (fast-forward the dataloader).
 - ④ **Restart:** Resume training.

Distributed Training

- A 70B parameter model in BF16 requires 140GB just for the weights.
- AdamW optimizer states require an additional 280GB.
- A single NVIDIA H100 GPU has 80GB of VRAM.
- **Conclusion:** The model does not fit on one GPU. We must parallelize.

To train massive models, frameworks like Megatron-LM use 3D Parallelism:

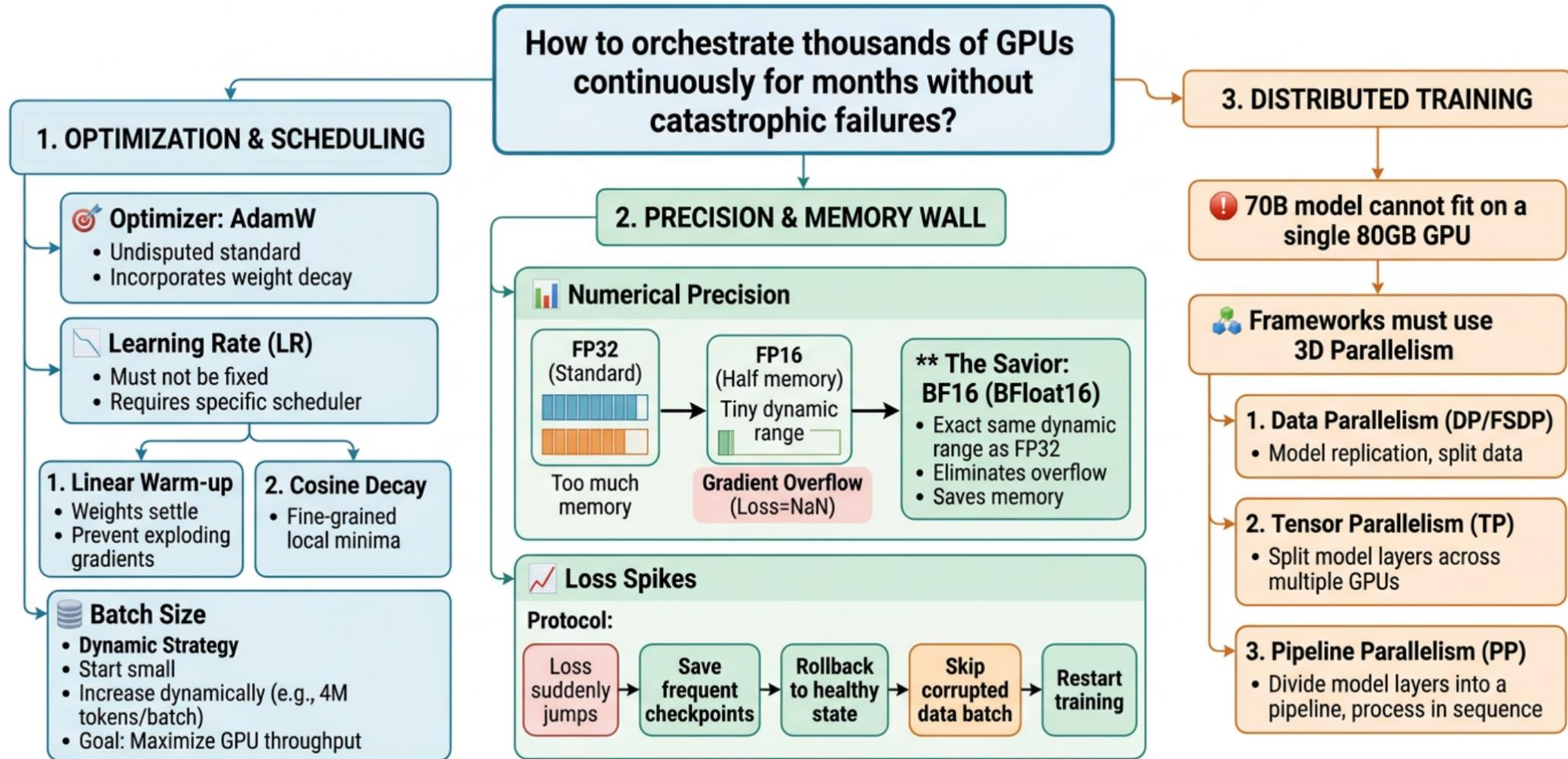
- 1 **Data Parallelism (DP / FSDP):** Replicate or shard the model across GPUs. Each GPU processes a different micro-batch of data.
- 2 **Tensor Parallelism (TP):** Split individual matrix multiplications (e.g., inside an Attention head) across multiple GPUs in the same node.
- 3 **Pipeline Parallelism (PP):** Assign layers 1-10 to GPU A, layers 11-20 to GPU B. Pass activations sequentially.

Parallelism

To train massive models, frameworks like Megatron-LM use 3D Parallelism:

- ① **Data Parallelism (DP / FSDP)**: Replicate or shard the model across GPUs. Each GPU processes a different micro-batch of data.
- ② **Tensor Parallelism (TP)**: Split individual matrix multiplications (e.g., inside an Attention head) across multiple GPUs in the same node.
- ③ **Pipeline Parallelism (PP)**: Assign layers 1-10 to GPU A, layers 11-20 to GPU B. Pass activations sequentially.

Module 4: Training Dynamic & Infra



Module 5: Utilizing the Base Model

How to use a Pretrained Base Model?

Once pretraining is done, how do we use the latent knowledge? (Min et al., 2023)

- ① **Fine-tuning:** Update weights for specific tasks.
- ② **Prompt-based Learning:** Use templates to align tasks with the pretraining objective.
- ③ **NLP as Text Generation:** Reformulate classification/parsing into pure text generation.

Prompt-Based Learning

- Reformat downstream tasks to look like the pretraining task.
- **Example (Sentiment):**
 - Input: "Best pizza ever!. It was ___"
 - LM predicts: "great" → Mapped to *Positive* class.
- **Why?** It exploits the specific MLM or CLM objective used during pretraining, leading to excellent zero-shot performance.

Proxy Tasks

- Convert a hard NLP task into a simpler "Proxy Task" the LM already knows.
- **Question Answering (QA) as a Proxy:**
 - Task: Named Entity Recognition (NER).
 - Prompt: "Which person is mentioned in the text?"
 - LM output identifies the entity.

NLP as Text Generation

- Generating structured outputs directly from the Decoder.
- **Label-Augmented Text:**
 - Input: "Tolkien's epic novel The Lord of the Rings..."
 - Output: "[Tolkien — person]'s epic novel [The Lord of the Rings — book]"
- Unifies all NLP tasks into a single Text-to-Text framework (like T5).

Conclusion

- **Data:** Trillions of heavily filtered, deduplicated tokens (Code + Web + Wiki).
- **Architecture:** Decoder-only Transformer, enhanced with RoPE, SwiGLU, and RMSNorm.
- **Math:** Scaled according to Chinchilla Laws (or overtrained for inference efficiency).
- **Engineering:** Trained using BF16, AdamW, Cosine Decay, and massive 3D parallelism.

Next Class: Post-Training

- How do we turn a "document completer" into a helpful Chatbot (like ChatGPT)?
- **Stage 2:** Supervised Fine-Tuning (SFT) - Teaching it the Q&A format.
- **Stage 3:** Reinforcement Learning from Human Feedback (RLHF) - Teaching it to be helpful, honest, and harmless.